

Definitive Guide to Immutable Cold Storage Architecture: Implementing a Multi-Tenant, Infinite-Incremental Optical Archival System for Linux

1. Executive Summary

The preservation of high-volume, static data assets presents a unique engineering challenge that stands in stark contrast to the requirements of high-availability transactional systems. The objective outlined is the creation of a "Write-Once, Read-Never" archival system for terabytes of Linux-hosted data, utilizing cold offline optical media (Blu-ray and M-Disc) as the storage substrate. The critical constraints—immutable incrementalism without full refreshes, optimization for file reorganization (renames/moves), strong cryptographic compartmentalization across distinct datasets, and the consolidation of disparate media—require a departure from traditional file-based backup paradigms.

Standard archival methodologies, such as those relying on tar, rsync, or linear tape file systems (LTFS), fail to meet the specific requirement of optimizing for file moves on write-once media. In a traditional model, renaming a directory necessitates re-writing the entire data payload to the new medium, a prohibitive cost when utilizing non-reclaimable optical discs. Furthermore, the requirement to mix encrypted datasets with distinct keys on a single physical medium while maintaining cryptographic isolation necessitates a sophisticated approach to repository management, decoupling the logical data structure from the physical storage blocks.

This research report proposes and details the **Linux Cold Storage Archival Suite (LCSAS)** architecture. This custom workflow leverages **Content-Defined Chunking (CDC)** to achieve storage-agnostic deduplication, ensuring that file moves and renames consume only negligible metadata space rather than duplicating payload storage.¹ The architecture utilizes a **Split-Horizon Repository Model**, utilizing **Rustic** (a high-performance Rust implementation of the Restic format) to maintain "hot" metadata on the server while ejecting "cold" encrypted data packs to optical media.²

To address physical media degradation ("bit rot"), the system integrates **DVDIsaster** for augmented error correction codes (ECC), wrapping the deduplicated data streams in a protective redundancy layer before burning.⁴ The consolidation of incremental backups is achieved through a "Migration and Pruning" protocol, which filters active data blobs from obsolete ones during the transfer from standard BD-R staging discs to permanent M-Disc archival media.

This document serves as an exhaustive technical manual for implementing, maintaining, and

recovering this system. It covers the theoretical underpinnings of CDC on WORM media, the cryptographic architecture for multi-tenant isolation, detailed shell scripting logic for the "burn-and-forget" workflow, and the physical handling requirements for M-Disc longevity. The analysis indicates that this approach provides the highest possible storage density and data durability for the specified constraints, effectively transforming commodity optical media into an enterprise-grade, ransomware-proof cold storage array.

2. Problem Space Analysis and Theoretical Constraints

The transition from magnetic storage (Hard Disk Drives, LTO Tape) to optical storage (Blu-ray, M-Disc) for terabyte-scale archives introduces strict physical limitations that software must overcome. Unlike magnetic media, which offers random access and rewrite capabilities, optical media is sequential, slow, and chemically immutable (in the case of HTL BD-R and M-Disc).

2.1 The Paradox of Write-Once Incrementalism

The primary challenge in the user's request is the requirement for "incrementals... optimized for moved/renamed files" without "ever refreshing a full backup."

In traditional backup theory, an "incremental" backup saves only files that have changed since the last backup. File change detection is typically performed via metadata comparison: checking the Modification Time (mtime) and file size.

- **The Rename Penalty:** If a user renames a folder containing 100GB of static video files from `/data/2023/vacation` to `/data/vacation_2023`, a metadata-based tool perceives every file within that tree as a *new* file. The path has changed; therefore, the file is different.
- **Consequence on WORM Media:** A standard tool like `tar` or `rsync` will re-read the 100GB of data and write a *new* 100GB session to the optical disc.⁵ On a hard drive, the old data could be deleted to reclaim space. On a write-once Blu-ray, that space is permanently lost. A simple organizational change could thus consume four 25GB discs, violating the efficiency requirement.

To solve this, the backup system must be **Path-Agnostic**. It must identify files by their *content*, not their location. This necessitates the use of Content-Defined Chunking (CDC), which fingerprints data streams to ensure that if the *content* exists anywhere in the repository history, it is never stored again, regardless of its filename or path.⁶

2.2 Cryptographic Compartmentalization on Shared Media

The requirement to "require strong encryption with different keys for distinct datasets" while "mixing small incrementals... on the same disc" introduces a conflict between isolation and efficiency.

- **Isolation:** Security best practices dictate that distinct datasets (e.g., "Friend" vs. "Personal") should not share encryption keys. If the "Friend" key is compromised,

"Personal" data must remain secure.⁷

- **Efficiency:** Optical media has fixed capacity (25GB, 50GB, 100GB). If "Friend" generates 2GB of new data and "Personal" generates 20GB, burning them to separate discs wastes vast amounts of space (23GB wasted on the "Friend" disc).
- **The Conflict:** Standard encrypted containers (like LUKS or Veracrypt) encrypt the entire volume with a single master key.⁸ We cannot burn a LUKS volume to a disc if it needs to contain data secure against two different keys.

The solution requires **Object-Level Encryption**. We must treat the optical disc not as an encrypted filesystem, but as a "dumb" container for encrypted objects. Each object (file chunk) must be individually encrypted with the key corresponding to its dataset *before* it is written to the disc. This allows objects from different security domains to coexist physically while remaining cryptographically distinct.⁹

2.3 The "Read-Never" Verification Challenge

Cold storage implies the media is disconnected. However, deduplication relies on knowing what is already stored. If the software cannot read the previous discs (because they are in a box on a shelf), how can it know that file image.jpg was already backed up three years ago? This necessitates a **Split-Horizon Architecture**:

1. **Hot Metadata:** The index of all stored chunks must remain live (online) on the server.
2. **Cold Payload:** The actual data chunks are ejected to the optical media. The backup software must be capable of trusting its local index without verifying the physical presence of the remote data during the backup phase. Most backup tools fail here, attempting to verify the integrity of the repo before writing. The chosen solution must support "blind" backups.²

3. Theoretical Architecture: Content-Defined Chunking and Split Horizons

To meet the efficiency requirements, we utilize the Restic repository format (specifically via the Rustic implementation), which fundamentally alters how data is addressed.

3.1 Content-Defined Chunking (CDC) Mechanics

CDC is the mechanism that allows for infinite incrementals and rename optimization. It processes the input stream using a rolling hash (typically a Rabin Fingerprint or a polynomial rolling hash) to detect boundaries in the data.¹

1. **Scanning:** The file is read as a continuous stream of bytes.
2. **Boundary Detection:** The rolling hash moves a sliding window across the bytes. When the hash value matches a specific pattern (mask), a "cut" is made. This creates a chunk.
3. **Identification:** The resulting chunk is hashed using a cryptographically strong algorithm (SHA-256). This hash becomes the chunk's ID.
4. **Deduplication:** The system checks its index.

- **Hit:** If the SHA-256 ID exists, the chunk is discarded. The system records only the ID in the file's metadata list.
- **Miss:** If the ID is new, the chunk is compressed, encrypted, and packed for storage.

Impact on Renames: When `/family/photo.jpg` is moved to `/family/archive/photo.jpg`, the content remains identical. The rolling hash produces the same chunks. The SHA-256 IDs are identical. The system looks up the IDs, finds them in the index (pointing to a disc burned 2 years ago), and writes **zero** new data packs. It simply creates a tiny metadata object linking the new path to the old chunks. This satisfies the "optimization for moved files" requirement perfectly.⁶

3.2 The Split-Horizon Topography

This architecture separates the repository into two distinct physical domains.

Component	Location	State	Content	Function
Hot Repository	Linux Server (SSD)	Read/Write	Config, Keys, Snapshots, Indices	Deduplication lookup, directory tree management, backup orchestration.
Cold Storage	Optical Media (BD-R)	Read-Only (WORM)	Data Packs (Encrypted Blobs)	Long-term payload retention.

This separation allows the server to perform "virtual" full backups. Every backup looks like a full backup to the user (browseable as a full file system), but physically consumes only the incremental space of new unique blocks. The rustic tool is uniquely suited for this as it allows specifying a `--repo-hot` and a separate repository target, essentially treating the optical media as a backend that is only written to, never read from, during the backup phase.³

3.3 Cryptographic Architecture: Multi-Tenant Blending

To satisfy the requirement of "different keys for distinct datasets" on the "same disc," we employ a **Multi-Repository, Single-Stage** approach.

We define three logical repositories:

1. Repo_Family: Initialized with Key A.
2. Repo_Personal: Initialized with Key B.
3. Repo_Friend: Initialized with Key C.

These repositories share nothing logically. They have separate indexes and separate deduplication domains (a file in Family will be stored again if it appears in Friend, which is desirable for security isolation).

However, they share a **Physical Staging Directory**.

- Repo_Family writes encrypted pack files (e.g., `a1b2...`) to `/mnt/staging/family/`.
- Repo_Personal writes encrypted pack files (e.g., `c3d4...`) to `/mnt/staging/personal/`.

The optical burning software (xorriso) treats these strictly as files. It burns the folder structure

of /mnt/staging/ onto the disc.

- **Result on Disc:** The disc contains folders family/, personal/, friend/.
- **Access Control:** The "Friend" user can mount the disc. They can see the *filenames* in the family/ folder (which are random hex strings). They cannot decrypt the contents because they lack Key A. The security requirement is satisfied via client-side encryption before the data ever touches the shared medium.¹³

4. Software Ecosystem Selection

The selection of open-source tools is critical for long-term archival. The tools must be stable, standard-compliant, and auditable.

4.1 Backup Engine: Rustic (vs. Restic)

While **Restic** is the established Go-based standard for CDC backups, **Rustic** (a Rust implementation) is the recommended engine for this specific project.

Why Rustic?

1. **Native Cold Storage Support:** Rustic introduces configuration options specifically for cold storage ([repository.options-cold]) and split repositories (--repo-hot). Restic requires cumbersome workarounds to achieve this, often forcing full index downloads or failing when data files are missing. Rustic is designed to operate with only the metadata online.²
2. **Configuration Profiles:** Rustic supports complex TOML configuration profiles. This allows defining the distinct encryption keys, repository paths, and cold storage parameters in persistent config files, reducing the risk of command-line errors during the complex multi-repo backup scripts.¹⁵
3. **Performance:** The Rust implementation offers significantly lower memory usage and faster cryptographic throughput, which is vital when processing terabytes of data on a potentially resource-constrained server.¹⁶

4.2 Optical Burning: Xorriso

Xorriso is the industry standard for advanced optical media manipulation in Linux. It supersedes mkisofs and cdrecord by combining their functionalities into a single manipulation environment.

- **Multi-Session:** It supports complex multi-session recording on BD-R, allowing us to add incrementals to a disc until it is physically full.¹⁷
- **Rock Ridge/Joliet:** It ensures full POSIX permission preservation (via Rock Ridge) while maintaining compatibility.
- **Scriptability:** It creates ISO streams on the fly, piping directly to the burning engine, avoiding the need to store temporary 25GB ISO files on the SSD (though for ECC generation, we will modify this approach slightly).¹⁸

4.3 Bit-Rot Protection: DVDDisaster (RS03)

Optical media degrades. Dyes fade; layers delaminate. **DVDDisaster** is non-negotiable for long-term archival.

- **RS03 Codec:** The Reed-Solomon (RS03) codec generates error correction data. Unlike PAR2 (which works on files), DVDDisaster works on the *image* (ISO) level.
- **Augmented Image:** We will use the "Augmented Image" mode. This pads the ISO image with ECC data filling the remaining space on the disc. If a sector becomes unreadable, DVDDisaster can mathematically reconstruct it using the redundancy data stored elsewhere on the disc.⁴
- **Superiority to PAR2:** PAR2 requires the file system to be readable to find the PAR files. If the directory table (UDF/ISO9660 structure) is corrupted, PAR2 fails. DVDDisaster can recover the raw image (and thus the directory structure) from raw sector reads.¹⁹

5. The LCSAS Workflow Specification

This section details the operational workflow for the **Linux Cold Storage Archival Suite (LCSAS)**.

5.1 Repository Initialization

We establish three repositories. We define a local directory `/mnt/staging` which represents the "Cold" storage target during the write phase. We define `/var/lib/rustic/` as the location for "Hot" metadata.

Directory Structure Setup:

Bash

```
mkdir -p /mnt/staging/{family,personal,friend}
mkdir -p /var/lib/rustic/{family,personal,friend}_hot
mkdir -p /root/keys
chmod 700 /root/keys
```

Initialization Commands:

We use `rustic init` with the `--repo-hot` flag. This separates the `config/index` (Hot) from the `data` (Cold/Staging).

Bash

```
# Initialize Family Repository
rustic init \
```

```
--repository /mnt/staging/family \  
--repo-hot /var/lib/rustic/family_hot \  
--password-file /root/keys/family.key \  
--use-config-file "profile_family.toml"
```

Repeat for Personal and Friend with respective paths and keys.

Critical Configuration Step:

The config file generated inside /mnt/staging/family/config contains the salt and chunker parameters. This file is **critical** and small. It must be copied to the Hot Metadata directory and backed up redundantly. If lost, the encrypted blobs are useless.

5.2 The Incremental Ingest (The "Daily" Job)

This process runs automatically. It identifies new data, chunks it, deduplicates against the Hot Index, and writes *only* new packs to the Staging area.

The "Cold" Logic:

Rustic will naturally look for existing data packs. Since previous packs have been burned to disc and deleted from staging, they are missing.

- **Configuration:** We must configure Rustic to *trust the index*. We use options effectively telling it: "Assume data exists; do not verify presence."
- In Rustic profiles, we set check-cache = false or ensure we rely on the hot-repo index without performing a backend check for existing blobs during backup.²

Script Logic (Pseudo-code):

1. **Mount Check:** Ensure the staging partition is mounted and has space.
2. **Loop Repositories:** Iterate through Family, Personal, Friend.
3. **Execute Backup:**
 - Command: rustic backup --repo-hot... --repository /mnt/staging/... /source/data
 - Rustic scans /source/data.
 - Calculates hashes.
 - Checks /var/lib/rustic/.../index.
 - If chunk exists (on a previous burned disc), it skips writing.
 - If chunk is new, it encrypts and writes to /mnt/staging/.../data/.
4. **Result:** /mnt/staging now contains a soup of new encrypted pack files from all three datasets.

5.3 The Burn Protocol (The "Weekly/Threshold" Job)

This process is triggered when the Staging Area size approaches the capacity of the target media (e.g., 23GB for a 25GB BD-R). This leaves ~2GB headroom for the filesystem overhead and ECC data.

The Integrated Burn Script:

This script performs the heavy lifting: creates the ISO, adds ECC, burns, and verifies.

1. **Manifest Generation:** Before burning, we index the Staging area.

Bash

```
find /mnt/staging -type f > /var/lib/rustic/catalogs/disc_manifest_$(date +%s).txt
```

This text file is crucial. It maps "Pack ID a1b2..." to "Disc created on Date X."

2. **ISO Creation (Xorriso):**

We do not write directly to disc yet. We create a temporary ISO image on the SSD.

Bash

```
xorriso -as mkisofs \  
-r -J -joliet-long \  
-V "LCSAS_ARCHIVE_$(date +%Y%m%d)" \  
-o /tmp/burn_image.iso \  
/mnt/staging
```

3. **ECC Augmentation (DVDDisaster):**

We apply the RS03 error correction *into* the ISO file.

Bash

```
dvdisaster -i /tmp/burn_image.iso -mRS03 -n 15 -c
```

Note: -n 15 specifies ~15% redundancy. On a 25GB disc, this allows recovering from substantial physical damage.

4. **Burning:**

Bash

```
xorriso -as cdrecord -v dev=/dev/sr0 -dao /tmp/burn_image.iso
```

5. **Verification:**

We use DVDDisaster to verify the burn quality immediately.

Bash

```
dvdisaster -i /dev/sr0 -t
```

6. **Purge:**

If verification passes:

- Delete contents of /mnt/staging/*.
- Update the "Cold Catalog" database, associating the manifest Pack IDs with the physical Disc Label (e.g., "DISC_001").

6. The Consolidation Protocol: Migration to M-Disc

The user requires "consolidating them from small discs to a single large M-Disc." This is a critical maintenance phase that also serves as "Offline Pruning."

6.1 The Challenge of Deletion on WORM

Since we cannot delete files from the BD-Rs, "deleted" files in the source system (e.g., a

deleted video) still exist as encrypted blobs on the BD-R backups. Over 5 years, a significant percentage of the BD-R capacity might be "dead" data (pruned snapshots). Consolidation is the only opportunity to reclaim this space.

6.2 The "Active Pack" Filter

We do not simply copy the ISOs. We perform a logical filter during the physical move.

Workflow:

1. Identify Active Data:

Run `rustic check --repo-hot...` on the server. This validates the metadata chain. Rustic knows which blobs are currently referenced by *active* snapshots.

We generate a list of **Active Pack IDs**:

Bash

```
rustic -r /mnt/staging/family --repo-hot /var/lib/rustic/family_hot list packs > /tmp/active_packs_family.txt
```

Repeat for Personal, Friend.

```
cat /tmp/active_packs_*.txt | sort | uniq > /tmp/all_active_packs.txt
```

2. Mount and Scrape:

Insert the small BD-R discs (Discs 1 through 10) sequentially.

Script logic:

Bash

```
MOUNT_POINT="/mnt/cdrom"
```

```
TARGET_STAGING="/mnt/mdisc_staging"
```

```
# User mounts Disc 1
```

```
for file in $MOUNT_POINT/family/data/*; do
```

```
    PACK_ID=$(basename "$file")
```

```
    if grep -q "$PACK_ID" /tmp/all_active_packs.txt; then
```

```
        cp "$file" "$TARGET_STAGING/family/data/"
```

```
    else
```

```
        echo "Skipping pruned pack: $PACK_ID"
```

```
    fi
```

```
done
```

Result: Only data that is *currently needed* is copied to the M-Disc staging area.

Obsolete data is left behind on the old BD-Rs.

3. Burn M-Disc:

Once `/mnt/mdisc_staging` reaches ~95GB (for a 100GB BD-XL M-Disc), execute the Burn Protocol (Section 5.3) with M-Disc parameters.

4. Catalog Update:

The "Cold Catalog" on the server is updated. The Pack IDs are now mapped to "MDISC_001". The old "DISC_001...010" entries are archived.

This satisfies the requirement to "avoid large size increases" by ensuring the long-term

storage is compacted and optimized.²⁰

7. Restoration and Disaster Recovery

A backup system is defined by its restore capability. Restoring from a split-horizon, multi-disc archive requires specific tooling.

7.1 The "Warm-Up" Strategy

Rustic fails if it tries to read a data blob that isn't present. We must "warm up" the data by copying it from the specific optical disc back to a staging area on the SSD.

The rustic restore Workflow:

1. **Identify Needed Packs:**

The user initiates a restore dry-run:

Bash

```
rustic restore --dry-run --json snapshot_id /target/path > restore_reqs.json
```

This output lists exactly which Pack IDs are required to reconstruct the requested files.²¹

2. **Locate Discs:**

A script parses `restore_reqs.json` against the local `cold_catalog.txt` database.

- Output: "You need to insert Disc 004, Disc 009, and M-Disc 001."

3. **Fetch Data:**

The user inserts Disc 004. The script copies the matching Pack IDs to `/mnt/staging/family/data`.

Repeat for all required discs.

4. **Execute Restore:**

Once all packs are present in the staging directory, the standard rustic restore command is run. It finds the data locally and decrypts it.

7.2 Total Loss Scenario (Server Destruction)

If the Linux server burns down, we lose the "Hot Metadata" (Indices and Keys).

- **The Risk:** Without the index, Rustic cannot know which encrypted blob belongs to which file.
- **The Mitigation:**
 1. **Key Backup:** The encryption keys (and Rustic config files) MUST be stored on a separate physical medium (e.g., a paper key or Cryptosteel, plus a USB drive at a relative's house).
 2. **Metadata Snapshots:** The "Hot Metadata" directory is small. We script a daily job to archive the `/var/lib/rustic` directory into a `metadata_backup.tar.gz.gpg` and burn this file to every optical disc we create in the ECC or meta folder.
 - *Result:* Every single optical disc contains a snapshot of the repository index as it existed at the time of burning. To recover, we grab the latest disc, restore the metadata directory, and we have the map. We then import the keys from our

8. Physical Layer Considerations: Media and Hardware

Implementing this system requires specific hardware choices to ensure validity over decades.

8.1 M-Disc Technology

Standard BD-R (HTL - High to Low) uses an inorganic phase-change alloy layer (typically copper/silicon or tellurium). This is far more stable than the organic dyes of CD-R/DVD-R.

M-Disc BD-R uses a patented glassy carbon layer that is rock-hard and chemically inert. It requires a higher-power laser to "engrave" the data.

- **Recommendation:** Use standard **Verbatim DataLifePlus HTL BD-R (25GB)** for the "Incremental/Staging" discs. They are cost-effective (~\$1/disc) and stable for 10-15 years.
- **Recommendation:** Use **Verbatim M-Disc BDXL (100GB)** for the "Consolidated" archives. These are expensive (~\$15/disc) but rated for centuries. This tiered approach balances cost and longevity.²³

8.2 Optical Drives

Not all drives can write M-Disc or BDXL.

- **Recommended Hardware:** Pioneer BDR-212UBK or LG WH16NS60. Both support BDXL and M-Disc burning. Pioneer is generally regarded as having superior error management and write quality.
- **Legacy Compatibility:** Standard Blu-ray drives can *read* M-Disc BD-Rs (unlike DVD M-Discs, which required specific readers). This ensures future readability.

8.3 Storage Environment

- **Darkness:** UV light is the enemy of optical media (though less so for M-Disc). Store in opaque boxes.
- **Temperature/Humidity:** Stable room temperature (20°C) and low humidity (40%).
- **Orientation:** Store discs vertically (like books), not stacked flat, to prevent warping over decades.

9. Implementation Blueprints (Scripting)

Below is the core logic for the **LCSAS Burner** script. This should be saved as `/usr/local/bin/lcsas-burn.sh`.

Bash

```

#!/bin/bash
# LCSAS - Linux Cold Storage Archival Suite - Burn Protocol
# Consolidates Staging Repos -> ISO -> ECC -> BD-R

STAGING_ROOT="/mnt/staging"
ISO_TEMP="/tmp/lcsas_buffer.iso"
LOG_FILE="/var/log/lcsas_burn.log"
DISC_LABEL="LCSAS_$(date +%Y%m%d)"

# 1. Safety Checks
if [ $(id -u) -ne 0 ]; then echo "Run as root."; exit 1; fi

# Calculate Size (in sectors for precision, here simplified to GB)
SIZE_GB=$(du -sBG "$STAGING_ROOT" | cut -d'G' -f1)
echo "Current Staging Size: ${SIZE_GB}GB"

# 2. Generate Catalogs (The Map)
echo "Indexing Staging..."
for repo in family personal friend; do
    if; then
        find "$STAGING_ROOT/$repo/data" -type f -printf "%f\n" >
"/var/lib/rustic/catalogs/${DISC_LABEL}_${repo}.txt"
    fi
done

# 3. Create ISO (Xorriso) with Rock Ridge (permissions) and Joliet (Windows compat)
echo "Generating ISO..."
xorriso -as mkisofs \
    -r -J -joliet-long \
    -V "$DISC_LABEL" \
    -o "$ISO_TEMP" \
    "$STAGING_ROOT" 2>>$LOG_FILE

if [ $? -ne 0 ]; then echo "ISO Creation Failed"; exit 1; fi

# 4. Apply DVDDisaster ECC (Augmented Image - RS03)
# -n 15 = 15% redundancy. Adjust based on free space.
echo "Applying RS03 Error Correction..."
dvdaster -i "$ISO_TEMP" -mRS03 -n 15 -c >>$LOG_FILE 2>&1

if [ $? -ne 0 ]; then echo "ECC Generation Failed"; exit 1; fi

# 5. Burn to /dev/sr0

```

```
echo "Ready to Burn. Insert BD-R and press Enter."
read -r _
```

```
xorriso -as cdrecord \
-v dev=/dev/sr0 \
-dao \
fs=64m \
"$ISO_TEMP"
```

6. Verify

```
echo "Verifying Burn with DVDDisaster..."
dvdaster -i /dev/sr0 -t
```

```
if [ $? -eq 0 ]; then
    echo "Verification SUCCESS."
    echo "Purging Staging Data..."
    rm -rf "$STAGING_ROOT"/*
    # Re-create skeletons
    mkdir -p "$STAGING_ROOT"/{family,personal,friend}/data
else
    echo "Verification FAILED. Do not delete staging."
    exit 1
fi
```

10. Conclusion

The architecture defined in this report satisfies every constraint of the user's request through a rigorous application of **Content-Defined Chunking** and **Cryptographic Abstraction**.

1. **No Full Refresh:** Rustic's CDC engine ensures that only new unique blocks are ever stored. The "full backup" is virtually reconstructed from the metadata index.
2. **Rename Optimization:** File moves generate zero payload data, merely updating the metadata index on the server.
3. **Isolation:** Three separate repositories with three distinct encryption keys share a physical medium. They remain cryptographically isolated because they are encrypted client-side before entering the shared staging area.
4. **Consolidation:** The migration to M-Disc acts as a garbage collection phase, physically copying only the active blobs identified by the Hot Index, effectively pruning deleted data from the permanent archive.

This system converts the Linux server into a sophisticated archival appliance. By decoupling the logical data view (Hot Metadata) from the physical storage (Cold Optical Discs), it achieves the data density and longevity of tape systems with the accessibility and low cost of

commodity optical media.

Table 1: Comparative Analysis of Archival Strategies

Feature	LCSAS (Proposed)	Traditional Tar/GPG	Encrypted Filesystem (LUKS)
Incremental Logic	Content-Based (CDC)	Time-Based (mtime)	Block-Level (Device)
Rename Cost	~0 GB	Full Duplicate (e.g. 50GB)	N/A (Image based)
Key Management	Per-Object (Granular)	Per-Archive (Tarball)	Per-Disc (All or Nothing)
Multi-Tenant Mixing	Native (Mixed Blobs)	Difficult (Separate Tars)	Impossible (One Key)
Bit-Rot Recovery	Integrated (RS03 ECC)	Manual (PAR2 sidecar)	None (FS Corruption)
Pruning	During Migration	Impossible (WORM)	Impossible (WORM)

1

Works cited

1. Foundation - Introducing Content Defined Chunking (CDC) - Restic, accessed February 12, 2026, <https://restic.net/blog/2015-09-12/restic-foundation1-cdc/>
2. FAQ - rustic user documentation, accessed February 12, 2026, <https://rustic.cli.rs/docs/FAQ.html>
3. Cold Storage - rustic user documentation, accessed February 12, 2026, https://rustic.cli.rs/docs/commands/init/cold_storage.html
4. speed47/dvdisaster: A tool providing additional ECC ... - GitHub, accessed February 12, 2026, <https://github.com/speed47/dvdisaster>
5. rsync - rsync when filenames have been changed identically on both ends - Super User, accessed February 12, 2026, <https://superuser.com/questions/1840227/rsync-when-filenames-have-been-changed-identically-on-both-ends>
6. Using dar for Data Archiving | The Changelog, accessed February 12, 2026, <https://changelog.complete.org/archives/10527-using-dar-for-data-archiving>
7. What is the point of multiple keys - restic forum, accessed February 12, 2026, <https://forum.restic.net/t/what-is-the-point-of-multiple-keys/4237>
8. Chapter 9. Encrypting block devices using LUKS | Security hardening | Red Hat Enterprise Linux | 8, accessed February 12, 2026, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/security_hardening/encrypting-block-devices-using-luks_security-hardening
9. Frequently asked questions — Borg - Deduplicating Archiver 2.0.0b21.dev27 documentation, accessed February 12, 2026, <https://borgbackup.readthedocs.io/en/latest/faq.html>

10. References — restic 0.18.1-dev documentation, accessed February 12, 2026, https://restic.readthedocs.io/en/latest/100_references.html
11. How to backup Linux, BSD and other Unix-like systems properly - Halfgaar's, accessed February 12, 2026, <https://www.halfgaar.net/backing-up-unix>
12. Rustic for file integrity checking / bitrot protection with cold storage backups? #1246 - GitHub, accessed February 12, 2026, <https://github.com/rustic-rs/rustic/discussions/1246>
13. rustic dev documentation, accessed February 12, 2026, <https://rustic.cli.rs/dev-docs/rustic-dev-docs.pdf>
14. Isolating backups between clients - Getting Help - restic forum, accessed February 12, 2026, <https://forum.restic.net/t/isolating-backups-between-clients/160>
15. rustic/config/README.md at main - GitHub, accessed February 12, 2026, <https://github.com/rustic-rs/rustic/blob/main/config/README.md>
16. Why should I switch from Restic to Borg? : r/BorgBackup - Reddit, accessed February 12, 2026, https://www.reddit.com/r/BorgBackup/comments/v3bwfg/why_should_i_switch_from_restic_to_borg/
17. xorriso - man pages section 1: User Commands, accessed February 12, 2026, https://docs.oracle.com/cd/E86824_01/html/E54763/xorriso-1.html
18. How to burn Blu-Ray disk from list of files using terminal? - Unix & Linux Stack Exchange, accessed February 12, 2026, <https://unix.stackexchange.com/questions/803330/how-to-burn-blu-ray-disk-from-list-of-files-using-terminal>
19. What Types of Data Could I still be using for CD-R's and DVD-R's in 2021? - Reddit, accessed February 12, 2026, https://www.reddit.com/r/DataHoarder/comments/quc78g/what_types_of_data_could_i_still_be_using_for/
20. Central catalog and backup spanning multiple disks - Getting Help - restic forum, accessed February 12, 2026, <https://forum.restic.net/t/central-catalog-and-backup-spanning-multiple-disks/10609>
21. `restore --warmup` option to enable cold storage based repositories · Issue #2796 - GitHub, accessed February 12, 2026, <https://github.com/restic/restic/issues/2796>
22. Disk Encryption: An Authoritative Guide for Linux Users, accessed February 12, 2026, <https://linuxsecurity.com/features/authoritative-guide-on-linux-disk-encryption>
23. Backup Solution? HDD vs M-DISC vs Other? | TrueNAS Community, accessed February 12, 2026, <https://www.truenas.com/community/threads/backup-solution-hdd-vs-m-disc-vs-other.87015/>
24. M-DISCs as long term(Shelf life) archival storage? : r/DataHoarder - Reddit, accessed February 12, 2026, https://www.reddit.com/r/DataHoarder/comments/zzmvay/mdiscs_as_long_terms

[help_life_archival_storage/](#)